PalmSource Developer Conference 2004
FEBRUARY 10-12, 2004 SAN JOSE, CA



Security on Palm OS® Cobalt and Prior Releases

The Evolution of Security on Palm OS®

Presented By:

Richard Levenberg

Security Architect

POWERED

The contents of and the technologies described in this presentation are provided to you and described "AS IS," without any warranty or representation of any kind. PalmSource hereby disclaims each and every other representation or warranty with respect thereto, including the implied warranties of merchantability, fitness for a particular purpose, accuracy and non-infringement. No representation or warranties are made that any product based in whole or in part on any specifications or information referenced herein will be free from defects or safe for use for its intended purposes.

PalmSource further reserves the right to modify in any respect any technology described or content provided herein without prior written notice.

Palm OS is a registered trademark and Palm Powered is a trademark of Palm Trademark Holding Company, LLC. Other brands may be trademarks of their respective owners. All other trademarks and tradenames used herein are the property of their respective owners.

Copyright (c) 2004 PalmSource, Inc.



Palm OS® Evolution

Versions

- **OS 1.x – 3.x**
- **OS 4.x**
- **OS 5.x (Palm OS® Garnet)**
- **OS 6.x (Palm OS® Cobalt)**



Palm OS® 1.x – 3.x

- **Encrypt.h provides MD5 and DES**
  - Very primitive but functional
- **Simplistic password encryption**
  - http://www.@stake.com/research/advisories/2000/index_q3.html#092600-1
- **Port of OpenSSL (SSLeay) crypto**
  - http://www.isaac.cs.berkeley.edu/pilot/
- **“Hidden” Certicom Security Builder on “7” devices (wireless security)**



Palm OS® 4.x

- **Fixed the password handling**
  - Obfuscation to MD5 (internal)
- **Mobile Internet Kit**
  - “Hidden” Ceriticom Security Builder
- **Certicom SecurityBuilder**
- **VPN**
  - IPSec (Certicom MovianVPN, SafeNet, SoftremoteVPN)
  - PPTP (Mergic)
- **RSA Wireless Core**
- **OnlyMe et.al**
  - http://www.tranzoa.com/html/compete.htm



Palm OS® Garnet

- **Cryptographic Provider Manager**
  - RSA implemented RC4, SHA1, RSA public key operations
  - Statically linked providers
  - FIPS 140-2 Level 1 Provider (certification pending)
- **VPN**
  - IPSec (Certicom MovianVPN)
  - PPTP (Mergic)
- **SSL**
  - RSA implemented server authenticated SSL v2, v3



Palm OS® Cobalt

- **CPM/SSL**
  - Pluggable providers, FIPS 140-2 Level 1 Provider (OS5 and 6)
  - SSL TLS 1.0, uses CPM
- **AuthenticationManager, Authorization Manager**
  - Pluggable authenticators
  - Arbitrary authorization
- **CertificateManager, SignVfy, PRCSign**
  - Meets signing requirements of OS
- **SecurityServices, PolicyCreator, DeviceSettings**
  - Allows licensees and user to specify policies and “paranoia” settings for the device


THE PLATFORM FOR MOBILITY
palm
POWERED



CPM/SSL

- **Pluggable Providers**
  - Original (5.x) static RC4, SHA1, RSA public
  - Legacy DES and MD5 provider
  - FIPS 140-2 Level 1 (SHA1, SHA2, AES, 3DES, HMAC)
- **SSL**
  - TLS 1.0, v3, v2
  - Uses CPM providers (SHA1, MD2, MD5, RSA)



CPM Example

```
uint8_t key[] = { 0x01, 0x23, 0x45, 0x67, 0x89, 0xAB, 0xCD, 0xEF };
uint8_t plain[] = { 0x01, 0x23, 0x45, 0x67, 0x89, 0xAB, 0xCD, 0xEF };
APKeyInfoType keyInfo;
MemSet(&keyInfo, sizeof(APKeyInfoType), 0);
keyInfo.type = apSymmetricTypeDES;
err = CPMLibImportKeyInfo(IMPORT_EXPORT_TYPE_RAW, key, sizeof(key),
&keyInfo);


// (continued on next slide)
```



CPM Example

```
// (continued from previous slide)
if (!err) {
  i = sizeof(data);
  err=CPMLibEncrypt(&keyInfo, NULL, plain, sizeof(plain), data, &i);
  if (err == cpmErrBufTooSmall) {
    uint8_t *outputP = MemPtrNew(output);
    if (outputP != NULL) {
      err = CPMLibDecrypt(&keyInfo, NULL, data, i, outputP, &output);
      MemPtrFree(outputP);
    }
  }
}
CPMLibReleaseKeyInfo(&keyInfo);
```



Authentication Manager (AM)

- **Pluggable authentication mechanisms (tokens)**
  - Password/Passphrase
  - Code fingerprint
  - PKI
  - *Biometric, other*
- **Authentication token "buckets"**
  - System defined (UserToken, SystemToken, AdminToken)
  - Application defined
  - "Bucket" contains a token against which authentication occurs



AM Example

```
AmTokenType token;
AmTokenInfoType tokenInfo;
Boolean bFoundToken = false;

AmGetTokenBySystemId(token, SysUserToken);
AmGetTokenInfo(*token, &tokenInfo);
if (tokenInfo.attributes.empty == 0) {
  do {
    err = AmAuthenticateToken(token, NULL, AmAuthenticationOther, “my
title”, “my message”);
    if (err == amErrUserCancel || err == amErrTokenDestroyed)
      break;
  } while (err);
}
```

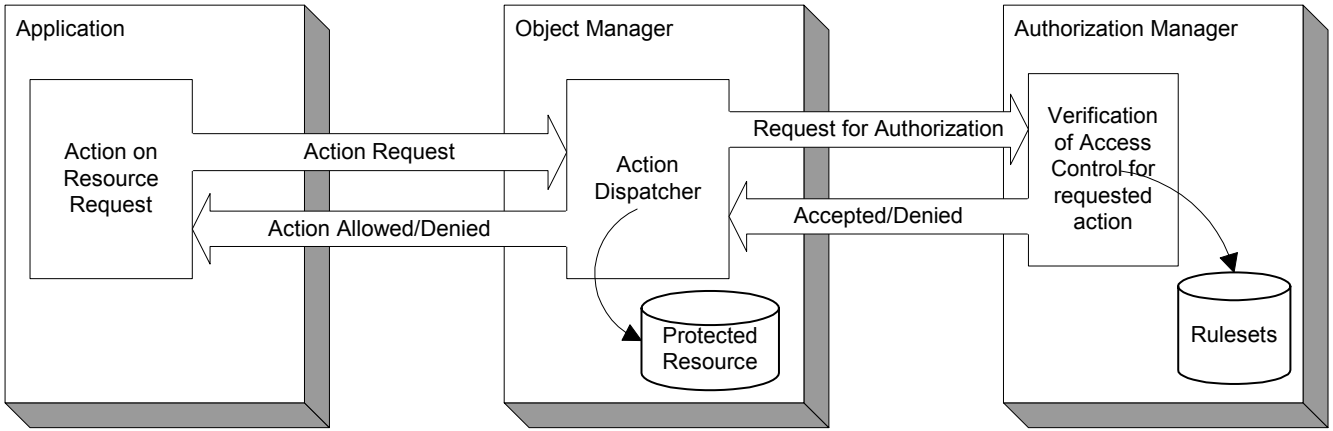


Authorization Manager (AZM)

- **Rules for access to arbitrary actions**
  - Actions defined by “manager” of protected object
- **Manager is responsible for defining action, calling AZM for attempted access**
- **Application, manager, client of manager sets up authorization “rules”**
- **Rules are associated with authentication tokens in AM**

Access Control and Authentication

- **Object Manager (DataMgr)**
  - Used by applications to create secure databases with specific access rules



- **Authorization Manager (AZM)**
  - Any managed resource can be protected with AZM
  - AZM handles all “actions” related to access rules



Secure Database (DataMgr & AZM)

- **DataMgr supports creation of secure databases**
  - DbCreateSecureDatabase
  - Only available with “new-style” databases
  - DataMgr defines actions (read, write, delete, backup, restore, etc.)
- **Application is handed back AZM ruleset (collection of rules)**
- **Application sets rules for actions**
  - E.g. AzmAddRule(ruleset, actions, authentication required, tokens)



Secure Database Example

```
AmTokenType token;
AzmRuleSetType azmRuleSet;


MemSet(&azmRuleSet, sizeof(AzmRuleSetType), 0);
DbCreateSecureDatabase(name, id, type, 1, schemaDefnP, &azmRuleSet,
NULL);


AmGetTokenBySystemId(token, SysUserToken);
AzmAddRule(azmRuleSet, dbActionRead|dbActionWrite, (char*)"%t", token);
```



Signing Infrastructure

- **Certificate Manager**
  - Handles X.509 certificates
  - Very basic handling (add, delete, get elements)
- **SignVfy Library**
  - Helper library for signature verification
- **PRCSign tool**
  - Signing tool to use with your own private keys



Signing Infrastructure (How it all works)

- **Use the signing tool to sign your application and set your shared library/overlay preferences**
  - Get a private key from somewhere
    - Self generate or get one from a CA
  - Get a certificate from somewhere
    - Self signed or again from a CA
  - Sign your application and decide if you want to set shared library and overlay protection
- **Applications may create secure databases and protect them with their PKI credentials**



Security Services

- **Device Policies (licensee)**
  - Licensee sets policies for the device
  - Policies are set for CPM plugins (export control)
- **Device Setting (user)**
  - High, Medium, Low (none)
- **Device Lockout**
- **PolicyCreator**
  - Java based tool that creates policies for SecurityServices



Device Policies

- **Processes**
- **FEPS and LocaleModules**
- **BSP Key**
- **IOS Drivers**
- **Sync clients**
- **AM Plugins**
- **CPM Providers (mandatory)**



Device Policies Example

```
  status_t err;
  MemPtr policyBuffer = NULL;
  uint32_t policyLength = 0;

  err = SecSvcsGetDevicePolicies('bspk', NULL, &policyLength);

  if (err == secSvcsErrBufferTooSmall) {
    policyBuffer = (uint8_t *)MemPtrNew(policyLength);
    if (policyBuffer != NULL) {
      SecSvcsGetDevicePolicies('bspk', policyBuffer,
&policyLength);
    }
  }
```



Device Setting (user paranoia)

SecSvcs(Get|Set)DeviceSetting

None, Medium, High

	None	Medium	High
Trusted Desktop	No UI	Ask user	Ask user
Sync Clients	No UI	Ask user	Ask user
Token Caching	Global	Application	Application
AM Plugin	No UI allow	Ask user	No UI deny



Device Setting example

```
status_t err;
SecSvcsDeviceSettingEnum setting;

err = SecSvcsGetDeviceSetting(&setting);

switch (setting) {
case SecSvcsDeviceSecurityNone:
  /* operate as if open device */
  break;

SecSvcsDeviceSecurityMedium:
  /* operate as if mildly paranoid */
  break;

SecSvcsDeviceSecurityHigh:
  /* operate as if paranoid */
  break;
}
```



Risks

- **Brand new operating system**
  - (If you don't already know what this implies…)
- **Sync/Backup**
  - Any "thing" that registers with SyncManager might get access to secure databases
  - Applications have to explicitly allow sync
    - AzmSetSyncBypass opens your database to any registered sync client
  - Backup is only permissible with UserToken protection
    - UserToken and only UserToken



Risks (continued)

- **Signed code**
  - If you sign your code and DO NOT add shared library and/or overlay protection your protected resource may be compromised
  - If you sign your code and DO add shared library and/or overlay protection, you may fail to operate in certain circumstances



Questions and Answers?

Richard Levenberg

Technical Security Architect

richard.levenberg@palmsource.com